

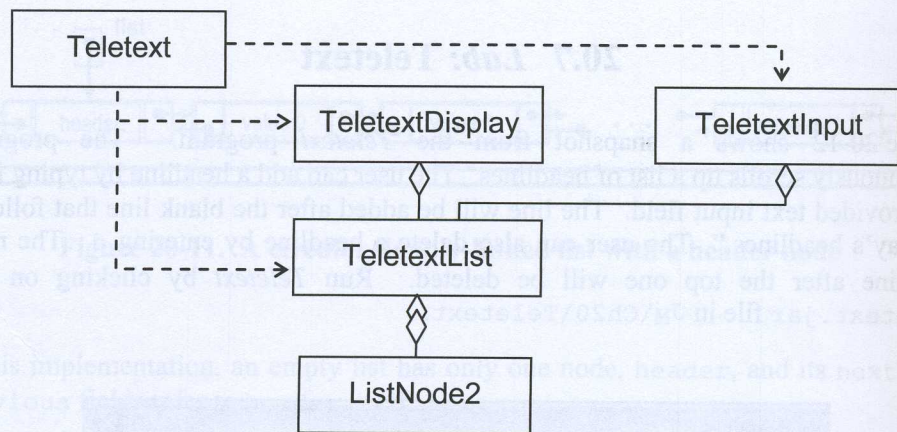


Figure 20-13. Class diagram for the *Teletext* program

20.8 Summary

In a *singly-linked list*, each element is stored in a larger structure, called a *node*. In addition to the element's value, a node contains a reference to the next node. The next reference in the last node is set to `null`. If `head` is a reference to the first node in the list, the statement

```
head = new ListNode(value, head);
```

appends a node holding `value` at the beginning of the list.

It is easy to traverse a linked list using a simple `for` loop:

```
for (ListNode node = head; node != null; node = node.getNext())
{
    Object value = node.getValue();
    ...
}
```

It is also not very hard to write a class that implements an “iterable” linked list for which iterators and “for each” loops work.

In a singly-linked list, inserting a node at the beginning takes $O(1)$ time, but appending a node at the end takes $O(n)$ time, where n is the number of nodes in the list. A *linked list with a tail* remedies this situation: it keeps a reference to the last